Achieving Max-min Fairness in Distributed Job Execution with Demand Transmissions

Shangyi Shao, Xueyan Tang
College of Computing and Data Science
Nanyang Technological University, Singapore
{shangyi001@e., asxytang@}ntu.edu.sg

***Abstract*—Deploying compute resources at multiple geographically distributed sites has become common practice for exploiting data locality to meet strict latency and reliability requirements of big data services. We study utilizing network resources to enhance the fairness of compute resource allocation among jobs by transferring data and hence computation demands among sites. We define a new resource allocation policy called Dynamic Aggregate Max-min Fairness (DAMF), which requires the aggregate compute resource allocation to be max-min fair among all jobs under given network capacity constraints. We prove that DAMF is achievable and it satisfies widely accepted fairness properties including Pareto efficiency, envy-freeness and strategy-proofness. We present algorithms to compute the DAMF allocation and efficient heuristics to approximate it. Experimental results show that compared with existing allocation policies, DAMF can improve the balance of resource allocation and achieve higher system utility, especially when the job's workload distributions among sites are highly unbalanced.**

I. Introduction

Geo-distributed processing has been prevalent for data analytics in various domains such as sensor networks [1], climate science [1], [2] and stock exchange [3]. In addition, it is increasingly common for data-processing jobs in geo-distributed systems to spread tasks among multiple sites for data locality: each task will be deployed and processed at some site where their input data are available [4]–[6]. Through this framework, the system can effectively save the bandwidth cost and improve the service response time. In such systems, providing fair resource allocation to jobs can help with preventing starvation and fulfilling system QoS, but it is a non-trivial problem. The problem consists of two critical hurdles of huge solution space: how to decide the distributed resource allocation for each job with various amount of resource demand distribution across sites, and how to utilize the limited network resources to transfer tasks of certain jobs between sites to achieve more fair allocation and better overall utility.

Previous work has explored the conventional max-min fairness in distributed job execution by requiring the aggregate resource allocation across all sites to be max-min fair [7]. However, if the job demand are distributed unevenly among the sites, the resulting resource allocation could be quite inefficient or unfair with respect to each job's demand. For example, if a high-demand job's entire workload is located at some site with small capacity, its allocation can still be unfair even if it takes up all resources at that site. However, to the best of our knowledge, achieving fair resource allocation exploiting the use of network resources has not been widely studied.

In this paper, we propose a resource allocation policy called Dynamic Aggregate Max-min Fairness (DAMF), which requires each job's aggregate resource allocation across all sites to be max-min fair among all possible demand transmissions and resource allocations. We prove that DAMF satisfies three important properties of fairness: Pareto efficiency, envy-freeness and strategy-proofness. We develop algorithms to compute the demand transmission and the DAMF resource allocation to each job at each site. Experimental results demonstrate that compared with existing allocation policies, DAMF can achieve resource allocations with better fairness among jobs and improve the system utility.

The rest of this paper is organized as follows. Section II describes the system model. Section III presents the preliminaries of max-min fairness. Sections IV and V introduce the definition of DAMF and its properties. Section VI presents algorithms for achieving DAMF. Experimental setup and results are discussed in Section VII. Section VIII summarizes the related work. Section IX concludes the paper.

II. System Model

Consider a geo-distributed system composed of m separate sites: $S_1, S_2, ..., S_m$. Each site S_j holds compute resources with capacity u_j. Suppose n jobs $J_1, J_2, ..., J_n$ are submitted in the system. Each job J_i consists of tasks that require distinct data partitions and could run in parallel. Due to the data locality consideration, initially each task is restricted to be served at a certain site where the input data is available. Each job's resource demand on a site is defined as the total demand of its tasks that can only run on this site. Let matrix D_{n*m} records the resource demands of all jobs across all sites, where each element d_{ij} denotes job J_i's demand on site S_j. Similarly, the resource allocation to all jobs on all sites can be represented by a matrix A_{n*m}, where each element a_{ij} denotes how much resource job J_i is allocated on site S_j. Throughout this paper, we shall use capital letters to stand for matrices and corresponding small letters to stand for the elements therein. For simplicity, we use $[x]$ to represent the set $\{1, 2, \ldots, x\}$ for a positive integer x.

Some basic constraints should be met to ensure that A_{n*m} is a sensible allocation. First of all, the summation of allocated

resources on any site should not exceed that site's capacity:

$$\forall j \in [m], \quad \sum_{i=1}^{n} a_{ij} \leq u_j. \tag{1}$$

Suppose the network in the geo-distributed system can be used to transfer data and hence demand from one site to another. Let matrix ΔD_{n*m} represent the demand transfers, where each element Δd_{ij} can be positive or negative: A positive Δd_{ij} describes how much demand of job J_i is transferred into site S_j, and vise versa. Apparently, we cannot transfer more demand out of a site than what a given job has at that site:

$$\forall i \in [n], j \in [m], \quad d_{ij} + \Delta d_{ij} \geq 0. \tag{2}$$

To model the bandwidth constraint, we assume a limit c_j at each site S_j, which represents an upper bound on the total demand that can be transferred into and out of site S_j:

$$\forall j \in [m], \quad \sum_{i=1}^{n} |\Delta d_{ij}| \leq c_j. \tag{3}$$

c_j can be estimated based on the available bandwidth and the average input data size of tasks. In the theoretical model, we assume that demand transfers are instantaneous. Our experimental evaluation will consider the transfer delay.

The total demand of each job should be kept after the transfer:

$$\forall i \in [n], \quad \sum_{j=1}^{m} \Delta d_{ij} = 0. \tag{4}$$

Note that upon transfers, the demand matrix becomes $D + \Delta D$. Since any job's allocation on any site should not exceed its demand on that site so that no resources are wasted:

$$\forall i \in [n], j \in [m], \quad 0 \leq a_{ij} \leq d_{ij} + \Delta d_{ij}. \tag{5}$$

We study the problem of defining and deriving a fair resource allocation A_{n*m} among jobs given the demands D_{n*m}, site capacities u_j's and bandwidth limits c_j's. The key notations used in this paper are summarized in Table I.

Table I: Notation Table

Notation	Definition
$\mathcal{M} = \{S_1, S_2, ..., S_m\}$	Set of sites
$\mathcal{J} = \{J_1, J_2, ..., J_n\}$	Set of jobs
u_j	Site S_j's compute resource capacity
c_i	Site S_j's bandwidth limit
D_{n*m}	Demand matrix
ΔD_{n*m}	Transmission matrix
A_{n*m}	Allocation matrix
$\mathcal{X}$	Set of all allocation matrices
X	Set of all aggregate allocation vectors

III. Preliminaries

A. Max-min Fairness

Max-Min Fairness (MMF) is one of the most common resource allocation strategies, which takes "serving the most unsatisfied user" as priority. Considering the resource allocation among n jobs as an n-dimensional vector, MMF can be defined as follows [8]:

Definition 1. *Consider a set V of n-dimensional vectors. A vector $\vec{x} \in V$ is max-min fair on set V, if and only if for any $\vec{y} \in V$, if $y_s > x_s$ for an index $s \in \{1, 2, ..., n\}$, there must exist another index $t \in \{1, 2, ..., n\}$ such that $y_t < x_t \leq x_s$.*

The max-min fair vector is unique if it exists, because if there are multiple max-min fair vectors within a given vector set V, we can derive contradiction simply by applying the definition of MMF to them. Clearly, the max-min fair vector does not exist in every vector set. Previous work has proved sufficient conditions for the existence of the max-min fair vector [8].

Theorem 1. *If a set V is compact and convex, then it is max-min achievable.*

B. Aggregate Max-min Fairness

A naive idea of applying MMF to distributed job execution is to consider each site independently and find the MMF resource allocation among jobs on each site based on their local demands. This policy is known as Independent Max-min Fairness (IMF) [7]. However, IMF can be far away from providing fair services to jobs from the global perspective due to the possibly unbalanced distribution of job demands and compute capacities among sites.

Guan *et al.* [7] defined Aggregate Max-min Fairness (AMF) for distributed job execution, which treats the distributed system as a united resource pool and constructs an aggregate allocation vector to represent each job's total resource allocation across all distributed sites:

$$\vec{A} = \langle \sum_{j=1}^{m} a_{1j}, \sum_{j=1}^{m} a_{2j}, ..., \sum_{j=1}^{m} a_{nj} \rangle.$$

AMF requires this vector to be max-min fair among all possible aggregation vectors. AMF does not consider utilizing network resources to transfer job demands between sites. Transferring job demands has the potential to improve the fairness of compute resource allocation for jobs as well as the system utility. In this paper, we study fair resource allocation among jobs with the use of network resources.

IV. Dynamic Aggregate Max-min Fairness

We propose a new resource allocation policy called Dynamic Aggregate Max-min Fairness (DAMF) to improve the fairness of resource allocation by taking the bandwidth availability across sites into consideration. Given the demand matrix, site capacities and bandwidth limits, there are a variety of possible transmission matrices and each would give rise to a large number of feasible and rational allocations. Under this circumstance, DAMF requires the aggregate allocation vector to be max-min fair among all the vectors derived from the allocation matrices resulting from all possible transmission matrices. Let $\mathcal{X}$ be the set of all feasible and rational allocation

matrices under all possible transmission matrices, and X be the set of all aggregate allocation vectors drawn from $\mathcal{X}$. DAMF can be defined as follows:

Definition 2. *An allocation A_{n*m} satisfies Dynamic Aggregate Max-min Fairness, iff its aggregate allocation vector $\vec{A}$ is max-min fair over the set X.*

The existence of DAMF requires the convexity and compactness of the set X according to Theorem 1. Due to space limitation, the proof of Theorem 2 can be found in the supplemental material, available at [9].

Theorem 2. *Dynamic Aggregate Max-min Fairness is achievable.*

V. Properties of DAMF

In this section, we study whether DAMF satisfies the properties of Pareto efficiency, envy-freeness and strategy-proofness, which are well-accepted characteristics of fairness.

A. Pareto Efficiency

Pareto efficiency means that no further improvement can be achieved which makes at least one individual better off without making any other individual worse off [10]. This includes two aspects [11]: first, no resource required by any consumer is left idle; second, no consumer is allocated any resource that it does not need. In our distributed job execution context, suppose the DAMF vector and its corresponding allocation matrix are $\vec{A}$ and A_{n*m} respectively. Pareto efficiency means that for any other feasible and rational allocation matrix A'_{n*m} and its corresponding aggregate allocation vector $\vec{A}'$, if there is an index i such that the i-th component of $\vec{A}$ is larger than the i-th component of $\vec{A}'$, i.e., $\sum_{j=1}^{m} a'_{ij} > \sum_{j=1}^{m} a_{ij}$, there must be another index k such that the k-th component of $\vec{A}$ is smaller than the k-th component of $\vec{A}'$, i.e., $\sum_{j=1}^{m} a'_{kj} < \sum_{j=1}^{m} a_{kj}$.

Theorem 3. *DAMF is Pareto efficient.*

Proof. By definition, the DAMF vector is the max-min fair vector among all possible aggregate allocation vectors under all possible transmission matrices. For any other feasible and rational allocation matrix A'_{n*m} and its corresponding aggregate allocation vector $\vec{A}'$, if $\sum_{j=1}^{m} a'_{ij} > \sum_{j=1}^{m} a_{ij}$ for some job J_i, then based on Definition 1, there exists another job J_k such that $\sum_{j=1}^{m} a'_{kj} < \sum_{j=1}^{m} a_{kj} \leq \sum_{j=1}^{m} a_{ij}$. Hence, the theorem is proven. □

B. Envy-freeness

Envy-freeness means that no individual would envy another individual's allocation. More specifically, in our distributed job execution context, envy-freeness means that for any job J_l, its usable allocation will not increase if it is given any other job J_k's transmission and allocation.

Theorem 4. *DAMF is envy-free.*

Proof. Let $\vec{A}$ be the DAMF vector, and A_{n*m}, ΔD_{n*m} be its corresponding allocation and transmission matrices respectively. Assume on the contrary that they are not envy-free, that is, a job J_l will have its usable allocation increased if it is given another job J_k's transmission Δd_{kj}'s and allocation a_{kj}'s. Note that the transmission refers to the network bandwidth allocated, and J_l does not have to fully utilize the allocated bandwidth in order to optimize its usable allocation. Let r_{lj}'s be the actual transmission used by J_l, where for each site S_j, $|r_{lj}| \leq |\Delta d_{kj}|$ (transmission is within the allocated bandwidth), $d_{lj} + r_{lj} \geq 0$ (transmission is feasible), and $\sum_{j=1}^{m} r_{lj} = 0$ (the overall demand of J_l is unchanged after transmission). Then, the demands of J_l on all sites would become $d_{lj} + r_{lj}$'s. Thus, J_l's usable allocation on each site S_j is $\min\{d_{lj} + r_{lj}, a_{kj}\}$. If J_l's total usable allocation over all sites increases, it means that

$$\sum_{j=1}^{m} \min\{d_{lj} + r_{lj}, a_{kj}\} > \sum_{j=1}^{m} a_{lj}.$$

This implies two facts:

(i) By eliminating the $\min$ operator, we have

$$\sum_{j=1}^{m} a_{kj} > \sum_{j=1}^{m} a_{lj},$$

which means job J_k's aggregate allocation is larger than J_l's aggregate allocation in the DAMF vector.

(ii) There must be at least one site $S_{j'}$ such that $\min\{d_{lj'} + r_{lj'}, a_{kj'}\} > a_{lj'}$, which indicates $d_{lj'} + r_{lj'} > a_{lj'}$ and $a_{kj'} > a_{lj'}$. The former inequality means that J_l's demand on site $S_{j'}$ is not completely satisfied, and the latter inequality means that J_k's allocation on site $S_{j'}$ is larger than J_l's allocation on $S_{j'}$.

By (ii), from the DAMF allocation matrix A_{n*m}, we could increase $a_{lj'}$ by decreasing $a_{kj'}$ on site $S_{j'}$. Accordingly, J_l's aggregate allocation increases while J_k's aggregate allocation decreases. Note that by (i), J_k's aggregate allocation is larger than J_l's aggregate allocation in A_{n*m}. This implies that we can increase the l-th component of the DAMF vector $\vec{A}$ by decreasing its k-th component which is larger, and this contradicts the fact that $\vec{A}$ is the max-min fair vector among all possible aggregate allocation vectors. □

C. Strategy-proofness

Strategy-proofness means that no individual could achieve better allocation by lying about its demand. In our distributed job execution context, strategy-proofness requires that no job could get higher usable allocation by lying about its demand d_{ij}'s, even if this would cause the recomputation of the transmission matrix ΔD and the allocation matrix A.

Theorem 5. *DAMF is strategy-proof.*

Due to space limitation, the proof of Theorem 5 can be found in the supplemental material, available at [9].

VI. Achieving DAMF

A. Algorithm for Computing DAMF Allocation

Algorithm 1 shows details to compute the DAMF allocation. The basic idea is to simultaneously increase the allocations of all jobs. Once a job is completely satisfied, the algorithm

Algorithm 1 Computing DAMF Allocation

1: **Input**: A set of sites $\{S_1, S_2, ..., S_m\}$ with capacities and transfer limits; a set of jobs $\{J_1, J_2, ..., J_n\}$ with demand matrix D;
2: **Output**: A DAMF allocation matrix A; a transmission matrix ΔD;
3: **Initialization**: $A = \mathbf{0}$; $\Delta D = \mathbf{0}$; $\mathcal{J} \leftarrow \{J_1, J_2, ..., J_n\}$;
4: **while** $\mathcal{J} \neq \emptyset$ **do**
5: Solve the following linear program:
6: Maximize T, subject to:
7: $\forall J_i \in \mathcal{J} : \sum_{j=1}^{m} a_{ij} \geq T$;
8: $\forall J_i \notin \mathcal{J} : \sum_{j=1}^{m} a_{ij} = t_i$;
9: $\forall j \in [m] : \sum_{i=1}^{n} a_{ij} \leq u_j$;
10: $\forall i \in [n], j \in [m] : 0 \leq a_{ij} \leq d_{ij} + \Delta d_{ij}$;
11: $\forall j \in [m] : \sum_{i=1}^{n} |\Delta d_{ij}| \leq c_j$;
12: $\forall i \in [n] : \sum_{j=1}^{m} \Delta d_{ij} = 0$;
13: Let T_{max} be the outcome of the above linear program;
14: **for** each $J_l \in \mathcal{J}$ **do**
15: **if** $\sum_{j=1}^{m} d_{lj} = T_{max}$ **then**
16: $t_l \leftarrow T_{max}$;
17: $\mathcal{J} \leftarrow \mathcal{J} \setminus \{J_l\}$;
18: **else if** $\sum_{j=1}^{m} a_{lj} = T_{max}$ **and** $\forall j \in [m], a_{lj} = d_{lj} \vee \sum_{i=1}^{n} a_{ij} = u_j$ **then**
19: Check the feasibility of the following program:
20: $\sum_{j=1}^{n} a_{lj} > T_{max}$;
21: $\forall J_i \in \mathcal{J} \wedge i \neq l : \sum_{j=1}^{m} a_{ij} = T_{max}$;
22: $\forall J_i \notin \mathcal{J} : \sum_{j=1}^{m} a_{ij} = t_i$;
23: $\forall j \in [m] : \sum_{i=1}^{n} a_{ij} \leq u_j$;
24: $\forall i \in [n], j \in [m] : 0 \leq a_{ij} \leq d_{ij} + \Delta d_{ij}$;
25: $\forall j \in [m] : \sum_{i=1}^{n} |\Delta d_{ij}| \leq c_j$;
26: $\forall i \in [n] : \sum_{j=1}^{m} \Delta d_{ij} = 0$;
27: **if** the above program is infeasible **then**
28: $t_l \leftarrow T_{max}$;
29: $\mathcal{J} \leftarrow \mathcal{J} \setminus \{J_l\}$;
30: **return** $A_{n*m}, \Delta D_{n*m}$;

fixes its allocation and continues to increase the allocations of others. The algorithm terminates when all jobs are satisfied or all available resources are used up.

Each iteration of Algorithm 1 consists of two stages. In stage 1, we define a unified lower bound T and maximize it by increasing the aggregate allocations of all jobs in set $\mathcal{J}$ simultaneously, where $\mathcal{J}$ is the set of jobs whose aggregate allocations have not been fixed yet. Initially, $\mathcal{J}$ contains all jobs (line 3). The maximum T value is derived by solving a linear program which models the constraints of resource allocation introduced in Section II (lines 5-12), where t_i is the fixed aggregate allocation of the job $J_i \notin \mathcal{J}$.

Let T_{max} denote the maximum T value obtained in stage 1. In stage 2, we identify jobs whose aggregate allocations are to be fixed. If a job J_l's total demand is equal to T_{max} amount of resources, it is fully satisfied. Thus, the algorithm fixes its aggregate allocation to be T_{max} and removes it from $\mathcal{J}$ (lines 15-17). Otherwise, the algorithm checks whether it is possible to increase J_l's aggregate allocation beyond T_{max} without violating other jobs' allocations. If J_l's aggregate allocation from stage 1 is already larger than T_{max}, or there exists a site on which J_l's demand is not fully satisfied and the site has spare capacity (i.e., the condition in line 18 is not satisfied), clearly J_l's aggregate allocation can be further increased. Otherwise, we change the aggregate allocation constraints for J_l and other jobs by replacing "$\geq$" with "$>$" (line 20) and "$=$" (line 21) respectively, and then check whether the new linear program has any feasible solution (lines 19-26). If so, it implies that J_l's allocation can be increased and J_l will remain in set $\mathcal{J}$. Otherwise, J_l's aggregate allocation is fixed at T_{max} and J_l is removed from set $\mathcal{J}$ (lines 27-29). Note that this T_{max} value is the aggregate resource amount, which means that J_l's allocations on individual sites could vary in subsequent iterations as long as the aggregate amount is kept. The algorithm terminates when $\mathcal{J}$ becomes empty (line 4).

The linear program in Algorithm 1 has $2mn$ variables and $2m+2n+mn$ constraints, where m and n are the numbers of sites and jobs respectively. Since there can be up to n iterations and each iteration needs to solve up to n linear programs in stage 2, the worst-case time complexity of Algorithm 1 is $O(n^2 * LP(2mn, 2m + 2n + mn))$, where $LP(2mn, 2m + 2n + mn)$ is the complexity for solving a linear program with $2mn$ variables and $2m + 2n + mn$ constraints.

B. A Fast Approximation

Algorithm 1 computes an exact DAMF allocation. However, its complexity could raise concern since solving large-scale linear programs can be computationally expensive. In this section, we propose a heuristic algorithm to derive a near-DAMF allocation with significantly reduced time complexity to enhance the practicality.

The heuristic algorithm (Algorithm 2) proceeds in two steps. In step 1 (lines 4-9), we find a fast approximation to AMF (i.e., without transferring job demands between sites) in an iterative manner. Each iteration calculates the compute resource allocation for one site S_j, given the job demands and compute capacity on this site as well as the aggregate allocation vector $\vec{p}$ over the sites that have been processed in previous iterations (line 5). Let $\vec{p} = \langle p_1, p_2, ..., p_n \rangle$ be the current aggregate allocation vector and $\langle d_{1j}, d_{2j}, ..., d_{nj} \rangle$ be the job demands on site S_j. We would like to compute an allocation $\langle a_{1j}, a_{2j}, ..., a_{nj} \rangle$ on S_j so that $\langle p_1+a_{1j}, p_2+a_{2j}, ..., p_n+a_{nj} \rangle$ is max-min fair among all possible allocations for the given $\vec{p}$. If the total demand of all jobs on site S_j is no larger than its capacity, i.e., $\sum_{i=1}^{n} d_{ij} \leq u_j$, we can simply satisfy the demands of all jobs by setting $a_{ij} = d_{ij}$ for all i's. Otherwise, we sort the jobs in increasing order of p_i, i.e., $p_1 \leq p_2 \leq ... \leq p_n$, and the resources on site S_j will be allocated to a prefix of this job sequence by the definition of max-min fairness. If resources are allocated to the first k jobs, the total resources allocated must be no more than

$$\sum_{i=1}^{k} \left(\min\{p_{k+1}, p_i + d_{ij}\} - p_i \right), \tag{6}$$

Algorithm 2 Heuristic to Approximate DAMF Allocation

1: **Input**: A set of sites $\{S_1, S_2, ..., S_m\}$ with capacities and transfer limits; a set of jobs $\{J_1, J_2, ..., J_n\}$ with demand matrix D;
2: **Output**: An allocation matrix A; a transmission matrix ΔD;
3: **Initialization**: $A = \mathbf{0}$; $\Delta D = \mathbf{0}$; $\vec{p} = \mathbf{0}$; sort all sites in increasing order of the number of jobs with demands on each site (i.e., the cardinality of $\{J_i : a_{ij} > 0\}$);
4: **for** each site S_j in the sorted order **do**
5: Compute max-min fair allocation $\langle a_{1j}, a_{2j}, ..., a_{nj}\rangle$ on S_j based on job demands $\{d_{1j}, d_{2j}, ..., d_{nj}\}$, capacity u_j of S_j, and current aggregate allocation vector $\vec{p} = \langle p_1, p_2, ..., p_n\rangle$ as described in the main text;
6: **for** $i \in [n]$ **do**
7: $p_i = p_i + a_{ij}$;
8: $\mathcal{S}_{spare} = \{S_j \mid \sum_{i=1}^{n} a_{ij} < u_j\}$;
9: $\mathcal{S}_{full} = \{S_j \mid \sum_{i=1}^{n} d_{ij} > u_j\}$;
10: $\mathcal{J} = \{J_i \mid \sum_{j=1}^{m} d_{ij} > \sum_{j=1}^{m} a_{ij}\}$;
11: **while** $\mathcal{S}_{spare} \neq \emptyset \wedge \mathcal{S}_{full} \neq \emptyset \wedge \mathcal{J} \neq \emptyset$ **do**
12: Remove all S_l's from $\mathcal{S}_{spare}$ and $\mathcal{S}_{full}$ that $c_l = 0$ or $\sum_{i=1}^{n}(d_{il} + \Delta d_{il}) = u_l$;
13: **while** $\mathcal{J} \neq \emptyset$ **do**
14: Select job $J_x \in \mathcal{J}$ with the least aggregate allocation;
15: **if** $\forall S_j \in \mathcal{S}_{full}, d_{xj} + \Delta d_{xj} \leq a_{xj}$ **then**
16: Remove J_x from $\mathcal{J}$;
17: **else**
18: Select a site $S_y \in \mathcal{S}_{full}$ where $d_{xy} + \Delta d_{xy} > a_{xy}$;
19: Select a site $S_z \in \mathcal{S}_{spare}$;
20: $\Delta d_{xy} = \Delta d_{xy} - 1$;
21: $\Delta d_{xz} = \Delta d_{xz} + 1$;
22: $a_{xz} = a_{xz} + 1$;
23: $c_y = c_y - 1$;
24: $c_z = c_z - 1$;
25: Break;
26: **return** $A_{n*m}, \Delta D_{n*m}$

because the aggregate resource allocation of each job cannot exceed the given allocation p_{k+1} of the next job ($(k+1)$-th job). Since (6) is non-decreasing with k, we can use a binary search to find out which jobs will receive resource allocations on S_j to make full use of its capacity u_j. After deciding k, we solve the equation $\sum_{i=1}^{k}(\min\{C, p_i + d_{ij}\} - p_i) = u_j$ to find out the maximum aggregate resource allocation C among these jobs to achieve max-min fairness. Then, the i-th job will be allocated $a_{ij} = \min\{C, p_i + d_{ij}\} - p_i$ amount of resources. Apparently, the iterating order of sites has an impact on the final allocation result. Our idea is to sort all sites in increasing order of the number of jobs with demands on the site (line 3). A site demanded by more jobs is considered to have higher potential in balancing the final aggregate allocation, so we process it in a later iteration.

In step 2 (lines 10-30), we utilize the network resources to improve the allocation derived in step 1. It is not difficult to see that there exists a DAMF allocation (represented by a pair of transmission and allocation matrices) such that for every demand unit (one task) transferred, (i) the task involved is allocated compute resources at the destination site; and (ii) the compute resources at the source site are fully allocated. In fact, if a demand unit transferred does not satisfy (i), the transfer can be canceled without affecting the allocation matrix. Similarly, the transfer of a demand unit that violates (ii) can also be canceled and the task can be satisfied by compute resources at the source site. This would increase the compute resource allocation of the job involved without affecting the compute resource allocation of any other job, thereby contradicting that the original allocation is max-min fair.

Based on the observation above, we define two sets $\mathcal{S}_{spare}$ and $\mathcal{S}_{full}$ to record sites with and without spare capacities (lines 10-11), and a set $\mathcal{J}$ to record jobs whose demands are not fully satisfied (line 12). Then, we start a loop to transfer job demands from $\mathcal{S}_{full}$ sites to $\mathcal{S}_{spare}$ sites until either $\mathcal{S}_{spare}$, $\mathcal{S}_{full}$ or $\mathcal{J}$ becomes empty (which implies that no further improvement to the allocation can be made). Intuitively, a site will not participate in transmissions if its bandwidth is used up or its updated local demand equals its capacity. Thus, within the loop, we remove all such sites from $\mathcal{S}_{spare}$ and $\mathcal{S}_{full}$ first (line 14). As a heuristic to approximate max-min fairness, among all jobs in $\mathcal{J}$, we pick the job J_x with the least aggregate allocation currently (line 16). If J_x's demands are satisfied on all sites in $\mathcal{S}_{full}$ (implying that J_x's unsatisfied demands locates at the sites that have been removed because no more bandwidth is available), we discard J_x and examine the next job with the least aggregate allocation for demand transmission (lines 17-18). Otherwise, we select the source site and destination site for a transfer of J_x: the source site can be any site in $\mathcal{S}_{full}$ where J_x's demand is not fully satisfied (line 20), and the destination site can be any site in $\mathcal{S}_{spare}$ (line 21). After deciding a transmission, we update the related matrices and remaining transfer limits (lines 22-26).

The heuristic algorithm does not solve linear programs and has much lower time complexity. In step 1, it solves m allocation problems separately (where m is the number of sites). In the water-filling-like strategy, sorting takes $O(n \log n)$ time (where n is the number of jobs), and the binary search takes $O(n \log n)$ time as computing (6) takes $O(n)$ time. So, the time complexity of each iteration is $O(n \log n)$, and the total time complexity of step 1 is $O(mn \log n)$. In step 2, the algorithm transfers at most $cm/2 = O(m)$ units of job demand (assuming the transfer limit for all site is a constant c). Updating $\mathcal{S}_{spare}$ and $\mathcal{S}_{full}$ takes $O(m)$ time. If we arrange all unsatisfied jobs in $\mathcal{J}$ as a min-heap based on their aggregate allocations, selecting the unsatisfied job with the least aggregate allocation takes $O(1)$ time. Selecting the source and destination sites takes $O(m)$ time. After the demand transfer, updating the min-heap of $\mathcal{J}$ takes $O(\log n)$ time. Thus, each unit of demand transfer takes $O(m + \log n)$ time. If

a source site for the selected job is found, the time complexity of step 2 is $O(m(m+\log n))$. Otherwise, the job is discarded and updating the min-heap of $\mathcal{J}$ takes $O(\log n)$ time. In the worst case, discarding all such jobs incurs a time complexity of $O(n(m+\log n))$. Therefore, the worst-case time complexity of step 2 is $O((m+n)(m+\log n)) = O(mn\log n + m^2)$. In the experiments, we shall empirically evaluate and compare the computation times of Algorithms 1 and 2.

C. Deployment

The algorithm for computing the DAMF allocation can be deployed at a central scheduler of the distributed job execution system. The scheduler tracks the real-time resource capacities of the sites and the distributed demands of each job on all sites. Once an allocation is derived, the scheduler propagates the transmission matrix ΔD and the allocation matrix A to relevant sites, where job demands (task input data) would be transferred based on ΔD. The compute resource allocation at each site would follow the allocation matrix A. When a computing slot becomes available on a site, the site computes the "Hunger Degree" for jobs with pending tasks locally and selects the job with the lowest Hunger Degree to launch its task in the available slot. More specifically, the Hunger Degree of a job on a site is the ratio between the number of its tasks that are currently running (the number of computing slots occupied by this job now) and its DAMF allocation on this site (the number of computing slots that this job is supposed to occupy). A lower Hunger Degree indicates that the job is taking up less slots than expected and deserves more slot allocation. In particular, if a job's DAMF allocation at a site is zero, its Hunger Degree is set to infinity.

In distributed job execution, the demands of jobs change continuously because of job arrivals and task completions. In general, the completion of a single task would not significantly change the relative distribution of job demands and hence the resultant resource allocation [7]. Therefore, to reduce the computational overhead, allocations can be recomputed only when a new job arrives or all tasks of an existing job are completed on some site. Recall that in our model, there is a time threshold T which determines the limit c_j on the total demand that can be transferred into and out of each site S_j. This parameter T can be dynamically adjusted according to the job arrival rate. When new jobs arrive frequently, T can be shortened to align with the reduced time between recomputations; conversely, T can be lengthened if the job arrival rate decreases.

VII. Experiments

Simulation experiments are conducted to compare the fairness and efficiency between various allocation policies.

Table II: Dataset characteristics

Dataset	Average task duration	Average task count per job
Alibaba	155.68s	273.5 tasks
Google	512.25s	4.5 tasks

A. Data Sets

We use the job traces from Alibaba [12] and Google [13]. The Alibaba trace contains information on approximately 16 million tasks collected from 1300 machines over a period of 12 hours in August 2017. The Google trace records events (e.g., a task is scheduled, completed or aborted) from Google's eight Borg cells over the month of May 2019. We pre-process the raw traces to extract key information, including the arrival time, task count and task durations of each job. From each trace, a continuous sequence of 15,000 jobs is randomly selected to serve as the dataset driving our simulations.

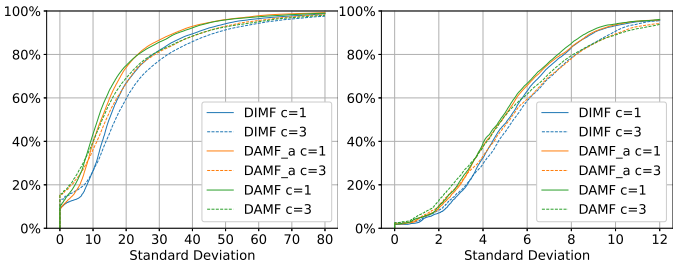
Table II shows the average task duration of the tasks of all jobs and the average task count per job in each dataset. We scale the job arrival times in both datasets to simulate different levels of overall system workloads.

B. System Configuration

We simulate a distributed system of m sites, each with 32 computing slots. Each task is assumed to take up one computing slot to run and release it after completion. We assume that there is no preemption during the execution of a task. For each job, the distribution of its tasks among the sites is generated by a Zipf distribution: the numbers of tasks placed on the sites are proportional to $1, \frac{1}{2^\alpha}, \frac{1}{3^\alpha}, ..., \frac{1}{m^\alpha}$, where α is a Zipf parameter. A higher α implies a more skewed distribution (a higher proportion of tasks are concentrated at one site), whereas α = 0.0 indicates uniform distribution. In the experiments, we test four Zipf distributions with α set to 0.0, 0.5, 1.0 and 2.0 to simulate different workload situations. We shuffle the order of the sites when deploying each job's task distribution, so that the overall workload distribution of all jobs is roughly balanced among sites. We experiment with different numbers of sites and different workload levels, and observe similar performance trends. Due to space limitations, we present the results for $m = 10$ sites and a system workload of 60% only.

C. Resource Allocation Policies

Besides Algorithms 1 and 2 (called DAMF and DAMF_a for short), we also implement three baseline policies: IMF, AMF (both introduced in Section IV) and Dynamic Independent Max-min Fairness (DIMF). DIMF utilizes network resources on top of IMF: it first transfers unsatisfied job demands to the sites with spare compute capacities to maximize the system utility (subject to the bandwidth limits) and then applies the IMF policy for resource allocation on each site. All the policies recompute the allocation matrix only when a new job arrives or an existing job completes all of its tasks on some site. We use CPLEX as the linear program solver in the experiments. In practice, demand transmissions must have integral values because a task is a basic unit for execution. We use probability sampling to round demand transmissions computed by Algorithm 1. For both datasets, the time to transmit the input data of one task is set to half of the average task duration, i.e., if the overall average task duration of a dataset is 100 seconds, then a transmission event will occupy



(a) Alibaba, workload = 60% (b) Google, workload = 60%

Figure 1: Cumulative distribution of standard deviation, Zipf parameter $\alpha = 2$

one bandwidth unit for 50 seconds (suppose each task has 1 GB input data, a bandwidth unit is about 160 Mbps). We test bandwidth limits of 1 and 3 units at each site. DAMF, DAMF_a and DIMF keep track of the current bandwidth usage and use the remaining bandwidth available at each site in recomputing the transmission and allocation matrices.

D. Performance Metrics

We study the following three performance metrics: the standard deviation of the instantaneous resource allocation among jobs, the average Jain's Index of instantaneous resource allocation, and the average job response time.

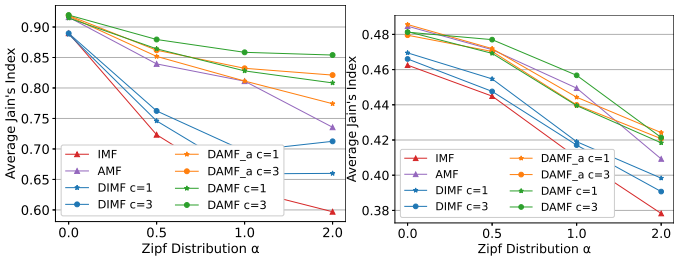
The standard deviation of the instantaneous resource allocation among jobs is computed from the records of the actual aggregate allocation vectors of jobs. We exclude jobs that have completed or not yet arrived from the computation of standard deviation. A lower standard deviation naturally indicates a more balanced resource allocation among jobs. We plot the cumulative distribution of the standard deviation over the simulated time span for comparison. Allocation policies whose distribution curves are closer to the upper left corner are considered superior in terms of instantaneous fairness because it has a lower deviation at higher percentage of times.

The Jain's Index is a quantitative measure used to assess the fairness or equity of resource allocation among n competing users or entities. Suppose each user i gets x_i resources, then Jain's Index can be computed as follows:

$$f(x_1, x_2, \ldots, x_n) = \frac{(\sum_{i=1}^{n} x_i)^2}{n * \sum_{i=1}^{n} x_i^2}. \tag{7}$$

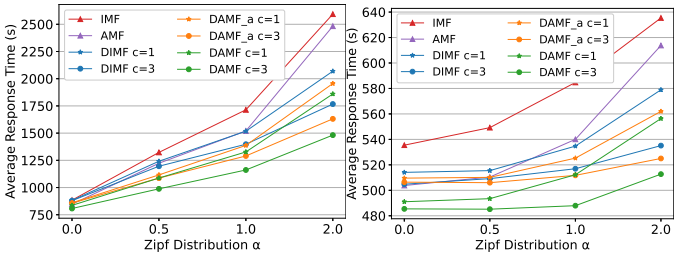
The range of Jain's Index is $[\frac{1}{n}, 1]$, and a higher index indicates a more balanced allocation. We compute the Jain's Index from the records of the actual aggregate allocation vectors of jobs, and calculate the average Jain's Index over the simulated time span for comparison.

The response time of a job is the time duration from its arrival to its completion. We use the average job response time to evaluate the long-term impact on efficiency by different allocation policies. A lower average job response time suggests higher system efficiency.



(a) Alibaba, workload = 60% (b) Google, workload = 60%

Figure 2: Average Jain's Index



(a) Alibaba, workload = 60% (b) Google, workload = 60%

Figure 3: Average job response time

E. Experimental Results

Figure 1 shows the cumulative distribution of the standard deviation of the instantaneous resource allocation among jobs. Due to space limitations, we again only present the results for the Zipf distribution of $\alpha = 2$. It can be seen that DAMF's distribution curves are closer to the upper left corner, which indicates that DAMF achieves more balanced instantaneous resource allocations than DIMF. Comparing DAMF and DAMF_a under the same transfer limit, we can see that their distribution curves are quite close to each other, which again shows that DAMF_a is a close approximation of DAMF. It can also be observed from Figure 1 that the standard deviation tends to increase with the transfer limit. This is because in DAMF, small-demand jobs are prioritized to be fully satisfied, while large-demand jobs are more likely to have pending demands and tend to exploit idle compute resources at other sites by leveraging network resources. Therefore, transmissions often help large-demand jobs to obtain more resources while keeping the allocation to small-demand jobs unchanged, which leads to the rise of the standard deviation of the instantaneous resource allocation with increasing transfer limit.

Figure 2 shows the average Jain's Index under different datasets and Zipf distributions. It can be seen that allocation policies that utilize task transmissions (DAMF, DAMF_a, DIMF) have higher Jain's Indexes compared with their counterparts which do not (AMF, IMF). More importantly, DAMF achieves the highest Jain's Index while DAMF_a is close to it, demonstrating that they effectively reduce the imbalance of resource allocation among jobs in long term.

Table III: Average computation time (s)

		Alibaba, workload = 60%		Google, workload = 60%	
c	α	DAMF	DAMF_a	DAMF	DAMF_a
1	0.0	0.1127	$1.402 * 10^{-5}$	0.3355	$6.346 * 10^{-5}$
	0.5	0.1539	$1.440 * 10^{-5}$	0.3246	$6.084 * 10^{-5}$
	1.0	0.1170	$1.453 * 10^{-5}$	0.3060	$5.071 * 10^{-5}$
	2.0	0.0970	$1.401 * 10^{-5}$	0.3770	$3.817 * 10^{-5}$
3	0.0	0.1456	$1.690 * 10^{-5}$	0.3512	$7.205 * 10^{-5}$
	0.5	0.1476	$1.351 * 10^{-5}$	0.3432	$5.482 * 10^{-5}$
	1.0	0.1573	$1.424 * 10^{-5}$	0.3331	$5.178 * 10^{-5}$
	2.0	0.1324	$1.316 * 10^{-5}$	0.3854	$3.670 * 10^{-5}$

Table IV: Average computation time (s) of DAMF_a

		Alibaba, workload = 60%		
c	α	$m = 20$	$m = 50$	$m = 100$
1	0.0	$3.732 * 10^{-5}$	$7.321 * 10^{-5}$	$1.594 * 10^{-4}$
	0.5	$3.339 * 10^{-5}$	$6.835 * 10^{-5}$	$1.592 * 10^{-4}$
	1.0	$2.517 * 10^{-5}$	$6.675 * 10^{-5}$	$1.439 * 10^{-4}$
	2.0	$2.243 * 10^{-5}$	$5.335 * 10^{-5}$	$1.219 * 10^{-4}$
3	0.0	$3.644 * 10^{-5}$	$7.452 * 10^{-5}$	$1.639 * 10^{-4}$
	0.5	$3.089 * 10^{-5}$	$7.945 * 10^{-5}$	$2.174 * 10^{-4}$
	1.0	$3.035 * 10^{-5}$	$9.177 * 10^{-5}$	$2.428 * 10^{-4}$
	2.0	$2.786 * 10^{-5}$	$9.714 * 10^{-5}$	$2.868 * 10^{-4}$

Figure 3 shows the average job response time for different datasets and Zipf distributions. It can be seen that policies utilizing network resources achieve lower average job response time, compared with IMF and AMF. The improvement typically is more significant when the transfer limit becomes larger. Furthermore, since DAMF views the whole distributed system as a united resource pool and requires the aggregate allocation vector to be max-min fair among all possible transmissions and allocations, it outperforms DIMF and other allocation policies in most cases. The improvement is substantial even with only one bandwidth unit available, and it becomes more significant when the system workload is higher or the task distribution of jobs is more skewed among sites. In addition, DAMF_a has similar performance of average job response time to DAMF, demonstrating that it closely approximates the original DAMF.

We also compare the average computation time for recomputing allocation matrices during simulations. Table III shows the computation time of DAMF and DAMF_a under different Zipf distributions, and Table IV shows the computation time of DAMF_a using the Alibaba dataset under larger number of sites $m = 20, 50$ and 100. The results show that DAMF_a notably enhances the computational efficiency compared to DAMF (from the order of 10^{-1} seconds down to 10^{-5} seconds), and the time cost stays acceptable (10^{-4} seconds) even the distributed system has 100 sites, demonstrating its excellent scalability to practical environments.

VIII. Related Work

Max-Min Fairness (MMF) is a widely used resource allocation policy. Radunovic *et al.* Ghodsi *et al.* [10] proposed Dominant Resource Fairness (DRF) which decides a user's allocation by its dominant share, i.e., the maximum proportional share that the user is allocated among all resource types. Ghodsi *et al.* [14] addressed task placement constraints and presented Constrained Max-min Fairness, which recursively satisfies the users based on the order of dissatisfaction. Zahedi and Lee [15] solved the multi-resource allocation problem by proportionally allocating different resources based on the re-scaled Cobb-Douglas Utility, namely Resource Elasticity Fairness. Gutman and Nisan [11] further extended DRF with respect to the bundle measure and utility functions. Narayana and Kash [16] presents the Least Cost Product mechanism to solve the multi-resource allocation problem under limited agent demands. Our work also considers multiple types of resources, but we focus on utilizing network resources to enhance the fairness of compute resource allocation.

In geo-distributed systems, a job may have tasks spread over different sites. Hung *et al.* [17] presented a scheduling algorithm for distributed job execution, which selects the job with the least overall demand to run first in order to minimize the average response time of jobs. This allocation policy is efficient but could cause starvation for large-demand jobs. Huang *et al.* [18] and Chen *et al.* [19] studied the job scheduling problem in distributed cloud services and tried to achieve max-min fairness for jobs in terms of the job response time. Different from the above work, we aim to achieve max-min fairness in the instantaneous resource allocation among jobs across geo-distributed sites given the availability of network resources. Recent studies have also adopted deep reinforcement learning to search for effective resource allocation strategies [20], [21]. However, learning-based approaches have mainly focused on improving the efficiency of resource allocation (such as system utility, revenue and profit) rather than achieving the fairness of resource allocation.

IX. Conclusion

We have defined a new resource allocation policy called Dynamic Aggregate Max-min Fairness (DAMF) in distributed job execution with task transmissions. We prove that DAMF satisfies several widely accepted properties, including Pareto efficiency, envy-freeness and strategy-proofness. We present an algorithm for computing the DAMF allocation and a fast approximation. Experimental evaluations using real job traces from Alibaba and Google demonstrate that DAMF could effectively enhance the fairness of resource allocation among jobs and improve the system utility.

REFERENCES

[1] Radu Tudoran, Gabriel Antoniu, and Luc Bouge. Sage: Geo-distributed streaming data analysis in clouds. In *2013 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum*, pages 2278–2281. IEEE, 2013.

[2] Radu Tudoran, Alexandru Costan, Rui Wang, Luc Bougé, and Gabriel Antoniu. Bridging data in the clouds: An environment-aware system for geographically distributed data transfers. In *2014 14th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*, pages 92–101. IEEE, 2014.

[3] Benjamin Heintz, Abhishek Chandra, Ramesh K Sitaraman, and Jon Weissman. End-to-end optimization for geo-distributed mapreduce. *IEEE Transactions on Cloud Computing*, 4(3):293–306, 2014.

[4] Ninad Hogade and Sudeep Pasricha. A survey on machine learning for geo-distributed cloud data center management. *IEEE Transactions on Sustainable Computing*, 8(1):15–31, 2022.

[5] Mohammed Bergui, Said Najah, and Nikola S Nikolov. A survey on bandwidth-aware geo-distributed frameworks for big-data analytics. *Journal of Big Data*, 8(1):40, 2021.

[6] Bita Fatemipour, Zhe Zhang, and Marc St-Hilaire. A survey on replica transfer optimization schemes in geographically distributed data centers. *IEEE Transactions on Network and Service Management*, 2024.

[7] Yitong Guan, Chuanyou Li, and Xueyan Tang. On max-min fair resource allocation for distributed job execution. In *Proceedings of the 48th International Conference on Parallel Processing*, pages 1–10, 2019.

[8] Bozidar Radunovic and Jean-Yves Le Boudec. A unified framework for max-min and min-max fairness with applications. *IEEE/ACM Transactions on networking*, 15(5):1073–1083, 2007.

[9] Shangyi Shao and Xueyan Tang. Supplementary materials of achieving max-min fairness in distributed job execution with demand transmissions. https://github.com/Mr-FLAG/DAMF_ISPDC, 2026.

[10] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *Proceedings of NSDI*, pages 323–336, 2011.

[11] Avital Gutman and Noam Nisan. Fair allocation without trade. In *Proceedings of AAMAS*, pages 719–728, 2012.

[12] Alibaba. Alibaba cluster trace program, 2017.

[13] Google. Google clusterdata 2019 traces, 2019.

[14] Ali Ghodsi, Matei Zaharia, Scott Shenker, and Ion Stoica. Choosy: Max-min fair sharing for datacenter jobs with constraints. In *Proceedings of the 8th ACM European Conference on Computer Systems*, pages 365–378, 2013.

[15] Seyed Majid Zahedi and Benjamin C Lee. REF: Resource elasticity fairness with sharing incentives for multiprocessors. In *Proceedings of ACM ASPLOS*, pages 145–160, 2014.

[16] Sushirdeep Narayana and Ian A Kash. Fair and efficient allocations with limited demands. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 5620–5627, 2021.

[17] Chien-Chun Hung, Leana Golubchik, and Minlan Yu. Scheduling jobs across geo-distributed datacenters. In *Proceedings of ACM SOCC*, pages 111–124, 2015.

[18] Zhe Huang, Bharath Balasubramanian, Michael Wang, Tian Lan, Mung Chiang, and Danny HK Tsang. Need for speed: Cora scheduler for optimizing completion-times in the cloud. In *IEEE INFOCOM*, pages 891–899, 2015.

[19] Li Chen, Shuhao Liu, Baochun Li, and Bo Li. Scheduling jobs across geo-distributed datacenters with max-min fairness. *IEEE Transactions on Network Science and Engineering*, 6(3):488–500, 2018.

[20] Amin Mohajer, Javad Hajipour, and Victor CM Leung. Dynamic offloading in mobile edge computing with traffic-aware network slicing and adaptive td3 strategy. *IEEE Communications Letters*, 2024.

[21] Xiaonan Li, Haijun Zhang, Huan Zhou, Ning Wang, Keping Long, Saba Al-Rubaye, and George K Karagiannidis. Multi-agent drl for resource allocation and cache design in terrestrial-satellite networks. *IEEE Transactions on Wireless Communications*, 22(8):5031–5042, 2022.